

20250616日报

今日工作内容

1. 学习silk_song项目，初步上手需求开发：将英雄皮肤购买日志和道具流水所需字段的最近72小时的100条数据写入redis。

此需求的处理逻辑：一条一条消费kafka->读取到ovlog.Parser中->redis加锁读取历史信息->按照core.KV结构存储历史信息->加入当前key对应的信息->截断72小时&100条->序列化后写到redis。

所遇问题：由于最初不理解redis存储的键值对结构，导致把该需求理解成了key为字段，value为字段对应的值。其实key=openid&zoneid，来表示一个所查询的用户；而value=历史信息，也就是所需的过滤字段。

以下为主要当前的代码处理逻辑（后续还需修改迭代）：

```

1 func process(ctx context.Context, openId string, zoneId int64, ts int64, parser
  *ovlog.Parser) error {
2     forceLog := rainbow.GetConfig().IsDyeing(openId)
3     key := fmt.Sprintf("silk_song_Itemchange:plusredis:%v:%v", zoneId, openId) //
    构造key，表示一个user
4     if forceLog {                                     // 打印白名单
5         log.InfoContextf(ctx, key)
6     }
7     oldValue := &core.KvMap{}
8     // 读取redis时先加锁
9     locker := newRedisLocker(fmt.Sprintf("lock:silksongplusredis_Itemchange:"))
10    if err := locker.Lock(ctx); err != nil {
11        return err
12    } else {
13        defer locker.Unlock(ctx)
14    }
15    { // 先从redis中读取原有的历史字段信息
16        bytes, err := redis.Bytes(proxies.SilkSongPlusRedis.Do(ctx, "GET", key)) //
    读取当前用户的历史信息
17        if err != nil {
18            if err != redis.ErrNil {
19                return err
20            }
21        } else {
22            if err := proto.Unmarshal(bytes, oldValue); err != nil { // 序列化历史信息
23                log.ErrorContext(ctx, err)
24            }
25        }
26    }
27    elements := Elements{}
28    elements.UnmarshalPb(oldValue) // core.KvMap->Element
29    // 当前信息
30    e := &Element{}

```

当前只是完成了两个需求字段的过滤逻辑，而还未消费kafka进行测试，并对结果的合理性进行分析。